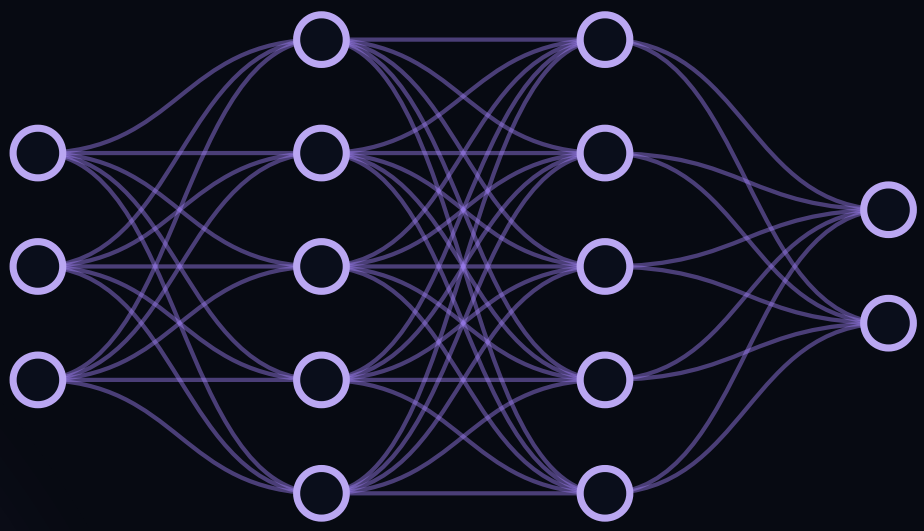
NeuralVarsity.ai

PROGRAM 03 · 6 MONTHS

Applied AI & Machine Learning

Engineer AI end to end — from tensors to autonomous agents.



6 Months	120	7	65
24 WEEKS	SESSIONS	PROJECTS	TOOLS

PROGRAM OVERVIEW

Engineer intelligent systems — from first principles to production.

An engineering-grade, 120-session program: production Python, statistics and classical ML, deep learning and computer vision with TensorFlow, Transformers built from scratch, advanced RAG and agentic systems, fine-tuning and alignment — then DSA, SQL and MLOps with FastAPI and Docker.

Who it's for

- Developers and engineers moving into AI engineering
- Data scientists going deeper into deep learning and LLMs
- Computer-science students and graduates
- Technical professionals building AI products

At a glance

- **Level:** Intermediate to advanced
- **Starting point:** Starts from Python foundations, then moves fast into engineering depth.
- **Format:** Day-by-day, engineering-grade
- **Generative AI:** Transformers from scratch, advanced RAG, LoRA/QLoRA, RLHF/DPO, LangGraph and MCP agents

What you'll be able to do

- Write production-grade Python — OOP, decorators, generators, concurrency, type hints, Pydantic and pytest
- Master statistics, feature engineering and every major classical ML family, with SHAP and LIME interpretability
- Build deep networks in TensorFlow: CNNs, ResNets, YOLO detection, segmentation and Vision Transformers
- Implement attention and a complete Transformer from scratch, and understand GPT-style decoding
- Architect advanced RAG — hybrid search, reranking, GraphRAG, CRAG — and multi-agent systems with LangGraph and MCP
- Fine-tune LLMs with LoRA and QLoRA and align them with RLHF and DPO
- Ship models with MLflow, FastAPI and Docker, and prepare for DSA and SQL interviews

Roles this program prepares you for

Machine Learning Engineer | AI Engineer | LLM / GenAI Engineer | Computer Vision Engineer | MLOps Engineer

AI Product Engineer

PROGRAM STRUCTURE

24 weeks · 6 months · 120 sessions

Month 01	**Python Foundations, OOP, & Data Engineering**	Days 1–20 20 sessions
Month 02	**Statistics, EDA & Foundational ML Models**	Days 21–40 20 sessions
Month 03	**Advanced ML, Deep Learning & Vision**	Days 41–60 20 sessions
Month 04	**NLP, Transformers & Generative AI Intro**	Days 61–80 20 sessions
Month 05	**Advanced GenAI, RAG, & Agentic Systems**	Days 81–100 20 sessions
Month 06	**Capstone Conclusion, DSA, SQL & MLOps**	Days 101–120 20 sessions

Tools & technologies

Python · VS Code · Jupyter · Pydantic · pytest · Requests · BeautifulSoup · Selenium · NumPy · Pandas · Matplotlib · Seaborn · Plotly · Dash · ydata-profiling · SciPy · scikit-learn · XGBoost · LightGBM · CatBoost · Optuna · SHAP · LIME · PyCaret · H2O.ai · Prophet · TensorFlow · Keras · Albumentations · YOLO (Ultralytics) · OpenCV · spaCy · gensim · Hugging Face · OpenAI API · Pinecone · ChromaDB · pgvector · Neo4j · LangChain · LlamaIndex · LangGraph · MCP · PEFT / TRL · CLIP · Streamlit · Gradio · n8n · Flowise · AWS Lambda · Google Cloud Run · Celery · Redis · RAGAS · DeepEval · LangSmith · GitHub Actions · Git · DVC · MLflow · FastAPI · Swagger UI · Postman · Docker · SQL

Month 01 — Python Foundations, OOP, & Data Engineering

Days 1–20

Day 1

Introduction to Programming & Environment Setup

Concepts: Differentiate between Algorithms (logic) and Programs (code).
Environment: Install Anaconda and configure VS Code/Jupyter.
Execution: Run your first interactive Python commands (print("Hello World")).
Virtual Environments: Introduce venv and conda to prevent package conflicts early on.

Day 2

Variables, Data Types & I/O

Syntax: Master Python indentation, variable naming rules, and comments.
Core Types: Understand int, float, str, and bool natively.
Inspection: Use type() to check objects and id() to see memory addresses.
I/O: Capture dynamic user input via input() and explicitly type-cast variables.

Day 3

String Manipulation & Formatting + Core Data Structures & Mutability

Indexing: Access characters and slice strings using [start:stop:step].
Methods: Apply .upper(), .lower(), .strip(), .replace(), and .split().
Formatting: Master modern f-strings for dynamic text injection.
Regex Intro: Use the re module for basic pattern matching and text normalization.
Lists & Tuples: Create ordered sequences. Compare mutable Lists vs. immutable Tuples.
Dicts & Sets: Implement key-value mappings and unique unordered sets.
Mutability: Understand why modifying a list changes it in memory, but modifying a string creates a new object.
Methods: Use .append(), .pop(), .keys(), and .items().

Day 4

Operators & Control Flow

Operators: Apply Arithmetic (+, %, //), Comparison (==, >), and Logical (and, or, not).
Conditionals: Build complex branching logic using if, elif, and else.
Loops: Iterate over data using for loops and manage state with while loops.
Control: Master break, continue, and pass statements within loops.

Day 5

Functions, Scope (LEGB) & Argument Unpacking

Definition: Create reusable logic blocks using def and return.
Arguments: Pass positional, keyword, and default parameters.
Flexibility: Build dynamic pipelines using *args and **kwargs.
Scope: Understand variable visibility via the LEGB rule.

Day 6

Functional Programming & Comprehensions + File I/O & Exception Handling

Anonymous: Write single-line logic using lambda functions.
Mapping: Transform and filter arrays cleanly using map() and filter().
Comprehensions: Replace heavy loops with optimized List, Dict, and Set comprehensions.
Efficiency: Compare the execution speed of loops vs. comprehensions.
Files: Read and write raw TXT and CSV files natively.
Context: Use the with open() statement to ensure safe resource teardown.
Errors: Prevent code crashes using try, except, else, and finally blocks.
Custom Errors: raise custom Exceptions for strict data validation.

Day 7

Object-Oriented Programming (OOP): Part 1

Blueprints: Define class structures and instantiate objects.
State: Initialize attributes using the __init__ constructor and self.
Inheritance: Pass behaviors from parent classes to child classes to avoid redundant code.
MRO: Understand Method Resolution Order in multiple inheritance.

Day 8

Object-Oriented Programming (OOP): Part 2

Encapsulation: Protect internal variables using name mangling (__attr).
Magic Methods: Override dunders (__str__, __len__, __add__) to make custom classes behave like native types.
Properties: Use @property decorators for controlled variable access.
Polymorphism: Ensure identical method names behave differently across subclasses.

Day 9

Decorators, Generators & Lazy Evaluation + CPython Internals & Memory Layout

Generators: Use yield to stream massive datasets without crashing RAM.
Decorators: Wrap functions with @ decorators to inject logic without altering core code.
Itertools: Build combinatoric pipelines using chain, cycle, and product.
Preservation: Use functools.wraps to keep function metadata intact.
Bytecode: Understand how Python compiles to .pyc and runs on the PVM.
References: Analyze memory pointers, reference counting, and garbage collection.
The GIL: Learn why the Global Interpreter Lock prevents true multi-threading in pure Python.
Profiling: Track memory leaks utilizing the tracemalloc library.

Day 10

Concurrency: Threading, Multiprocessing & Asyncio

Threading: Handle network/I/O tasks via ThreadPoolExecutor.
Multiprocessing: Bypass the GIL for heavy math utilizing ProcessPoolExecutor.
Asyncio: Build single-threaded, non-blocking loops using async and await.
Safety: Prevent race conditions using Locks and Semaphores.

Day 11

Advanced Packaging, Type Hints & Pydantic

Typing: Enforce strict parameter types using the typing module and mypy.
Validation: Ensure JSON data integrity natively using Pydantic schemas.
Boilerplate: Auto-generate class structures using @dataclass.
Modules: Structure large projects with __init__.py and pyproject.toml.

Day 12

NumPy: Arrays, Broadcasting & Vectorization + Pandas Fundamentals & Data Structures

Tensors: Create N-dimensional matrices (np.zeros, np.arange).
Vectorization: Execute C-level mathematical operations natively without slow Python loops.
Broadcasting: Perform arithmetic mathematically on arrays of mismatched shapes.
Linear Algebra: Compute dot products (np.dot) and matrix multiplications.
Series/DataFrames: Master 1D and 2D tabular data structures.
Importing: Ingest pd.read_csv, pd.read_json, and pd.read_sql.
Inspection: Assess datasets rapidly utilizing .head(), .info(), and .describe().
Selection: Filter columns and rows explicitly via .loc[] and .iloc[].

Day 13

Advanced Pandas: Cleaning & Aggregation

Missing Data: Impute or drop nulls using .fillna() and .dropna().
Transformations: Apply complex custom logic across rows via .apply().
GroupBy: Execute SQL-style aggregations utilizing .groupby() and .agg().
Dates: Parse and manipulate temporal features via the .dt accessor.

Day 14

Data Merging & Automated Web Scraping

Combining: Join disparate datasets using pd.merge() (Inner, Left, Right).
Requests: Fetch raw HTML securely via the requests HTTP library.
Parsing: Extract targeted data tags using BeautifulSoup.
Dynamic: Automate browser interactions for JS-rendered sites via Selenium.

Day 15

Code Testing & Logging + Capstone: End-to-End Extraction Pipeline

Unit Testing: Write robust test cases using the pytest framework.
Mocking: Simulate API responses to test functions in isolation.
Logging: Replace print() with the logging module (Info, Warning, Error, Critical).
Debugging: Master the pdb debugger to step through code execution dynamically.
Extraction: Write a scraper to collect raw data from a multi-page website.
Cleaning: Utilize Pandas to parse strings, handle missing values, and type-cast.
Architecture: Structure the code using strict OOP principles, Type Hints, and Decorators.
Output: Export the finalized, clean dataset to a SQLite database or Parquet file.

Day 16

Data Visualization with Matplotlib

Core API: Build layouts via plt.subplots() rather than the state machine.
Plot Types: Generate line, bar, scatter, histogram, and pie charts.
Customization: Apply labels, legends, colormaps, and annotations.
Export: Save high-res publication figures using plt.savefig().

Day 17

Data Visualization with Seaborn & Plotly

Seaborn: Plot distributions (boxplot, violinplot, heatmap, pairplot).
Plotly Express: Build interactive 3D scatters, animations, and maps.
Plotly Graph Objects: Gain fine-grained control over complex interactive plots.
Dash: Introduction to web-based interactive ML dashboard frameworks.

Day 18

Exploratory Data Analysis (EDA) Framework + Descriptive Statistics: Central Tendency & Dispersion

Univariate: Analyze shape, skewness, and kurtosis via histograms and KDEs.
Bivariate: Identify feature-target correlations using point-biserial and scatter plots.
Multivariate: Detect multicollinearity using Variance Inflation Factor (VIF).
Automation: Rapidly generate reports using the ydata-profiling library.
Centrality: Calculate mean, median, mode, and geometric mean.
Spread: Measure variance, standard deviation, range, and Interquartile Range (IQR).
Outliers: Apply Tukey's fences (Q1 − 1.5 × IQR, Q3 + 1.5 × IQR).
Dependencies: Compute covariance and Pearson correlation mathematically.

Day 19

Probability Foundations & Bayes' Theorem

Axioms: Define sample spaces, unions, intersections, and conditional probability.
Bayes' Theorem: Compute Posterior from Prior, Likelihood, and Evidence.
Distributions: Differentiate Discrete vs. Continuous random variables.
ML Application: Connect Bayes' to Naive Bayes classifiers and updating.

Day 20

Discrete & Continuous Probability Distributions

Discrete: Model outcomes using Bernoulli, Binomial, and Poisson distributions.
The Gaussian Curve: Master the Normal distribution and Z-score standardization.
Heavy Tails: Apply the Student's t-distribution for small-sample continuous data.
Computation: Generate PDFs/PMFs programmatically using scipy.stats.

Month 02 — Statistics, EDA & Foundational ML Models

Days 21–40

Day 21 — Central Limit Theorem & Sampling + Correlation Analysis & Feature Relationships

CLT: Prove that sample means converge to a Normal distribution regardless of shape.
Standard Error: Calculate precision using SE = σ/√n.
Bootstrapping: Generate non-parametric sampling distributions via resampling.
Strategies: Execute Random, Stratified, and Cluster sampling methodologies.
Parametric vs Non-Parametric: Compare Pearson's r against Spearman's rank ρ.
Categorical Links: Compute point-biserial and Phi coefficients.
Information Theory: Calculate Mutual Information for non-linear feature selection.
Causality: Identify spurious correlations and understand Simpson's Paradox.

Day 22 — Hypothesis Testing: Framework & p-values

Framework: Define the Null (H_0) and Alternative (H_1) hypotheses clearly.
Risk Matrix: Quantify Type I (False Positive) and Type II (False Negative) errors.
Metrics: Calculate p-values, significance levels (α), and statistical power (1-β).
Confidence Bounds: Construct 95% CIs using Z-values and t-values.

Day 23 — Statistical Testing: Parametric & Non-Parametric

Mean Comparisons: Execute Z-tests, independent T-tests, and paired T-tests.
Multi-Group: Run One-Way and Two-Way ANOVA utilizing the F-statistic.
Rank Tests: Use Mann-Whitney U and Kruskal-Wallis for non-normal data.
Categorical: Execute Chi-Square tests of independence and A/B test setups.

Day 24 — Intro to ML Taxonomy & The Bias–Variance Tradeoff + Feature Engineering & Scaling

Taxonomy: Define Supervised, Unsupervised, Semi-Supervised, and RL paradigms.
Workflow: Map the pipeline from EDA to Deployment and Monitoring.
Tradeoff: Mathematically decompose MSE into Bias², Variance, and Noise.
Mitigation: Counter underfitting/overfitting via regularization and ensembles.
Scaling: Normalize features via StandardScaler, MinMaxScaler, and RobustScaler.
Encoding: Apply One-Hot and Target encoding using get_dummies and scikit-learn.
Imbalance: Generate synthetic minority instances using SMOTE and ADASYN.
Pipelines: Automate workflows safely using ColumnTransformer and Pipeline.

Day 25 — Model Validation & Leakage Prevention

Splitting: Implement strict Train/Validation/Test splits with class stratification.
CV Methods: Execute k-Fold, LOOCV, and TimeSeriesSplit.
Data Leakage: Identify and prevent temporal violations and pre-split scaling.
Nested CV: Tune hyperparameters without optimistic bias via nested loops.

Day 26 — Linear Regression: OLS & Diagnostics

OLS Math: Solve regression weights using closed-form algebra.
Assumptions: Validate the Gauss-Markov theorems (linearity, homoscedasticity).
Diagnostics: Plot residuals vs. fitted values and calculate Cook's Distance.
Evaluation: Calculate MSE, RMSE, MAE, and Adjusted R^2 mathematically.

Day 27

Multiple Regression, Collinearity & Regularization + Logistic Regression & Decision Boundaries

Multivariate: Interpret partial derivative coefficients in multiple dimensions.
Collinearity: Detect feature correlation using VIF and mitigate via PCA.
Optimization: Implement Batch, Stochastic (SGD), and Mini-Batch Gradient Descent.
Regularization: Induce sparsity via L1 (Lasso) and shrink weights via L2 (Ridge).
Sigmoid: Squash linear outputs into probabilities using $\sigma(z) = 1/(1 + e^{-z})$.
Interpretability: Extract log-odds and convert to multiplicative Odds Ratios.
Optimization: Minimize Binary Cross-Entropy loss via Maximum Likelihood.
Implementation: Train via LogisticRegression using lbfgs and saga solvers.

Day 28

Classification Evaluation Metrics

Confusion Matrix: Map True/False Positives and Negatives.
Ratios: Calculate Precision (purity), Recall (completeness), and the F1-Score.
ROC-AUC: Evaluate threshold-independent performance using TPR vs. FPR.
Imbalance Tools: Plot Precision-Recall curves and calculate Matthews Correlation.

Day 29

Naive Bayes Classifiers & Text Applications

Bayesian Math: Apply conditional independence assumptions for rapid $O(n \cdot p)$ fitting.
Variants: Use GaussianNB for continuous data and MultinomialNB for NLP counts.
Smoothing: Implement Laplace additive smoothing to handle zero-probability features.
NLP Pipelines: Combine Naive Bayes with TF-IDF vectorization for text sorting.

Day 30

K-Nearest Neighbors (KNN) + Support Vector Machines (SVM)

Lazy Learning: Classify via majority vote without an explicit training phase.
Metrics: Calculate Euclidean, Manhattan (L1), and Cosine similarities.
Hyperparameters: Tune k using cross-validation to balance bias/variance.
Scaling: Accelerate search queries using KD-trees and Ball trees.
Hyperplanes: Maximize the geometric margin between separable classes.
Soft Margins: Introduce slack variables (ξ) to handle non-linearly separable data.
Kernel Trick: Map features to infinite dimensions via Linear, Polynomial, and RBF kernels.
Hyperparameter C: Tune the strictness of the margin constraint (bias vs variance).

Day 31

Decision Trees: CART, Information Gain & Splitting

Splitting Logic: Partition features by minimizing Gini Impurity or Entropy.
CART Algorithm: Perform exhaustive greedy search for optimal threshold splits.
Importance: Extract normalized feature importance scores based on impurity drops.
Visualization: Plot hierarchical splits utilizing sklearn.tree.plot_tree.

Day 32

Tree Pruning & Hyperparameter Tuning

Overfitting: Understand how unconstrained trees memorize noise.
Pruning: Constrain growth using max_depth, min_samples_leaf, and ccp_alpha.
Tuning Pipeline: Automate optimal parameter discovery using GridSearchCV.
Decision Boundaries: Visualize orthogonal, step-wise decision boundaries mathematically.

Day 33

Ensemble Methods Part 1: Bagging & Random Forests + Ensemble Methods Part 2: Boosting Foundations

Ensemble Theory: Combine weak learners to build a highly robust generalized model.
Bagging: Train independent models on bootstrapped data subsets.
Random Forests: Force spatial decorrelation via random feature subset selection.
OOB Scoring: Validate model performance dynamically using Out-of-Bag data metrics.
Boosting Theory: Train sequential models to explicitly correct predecessor residuals.
AdaBoost: Dynamically reweight misclassified instances to force mathematical attention.
Gradient Boosting: Fit trees directly to the negative gradients of any differentiable loss function.
Learning Rate: Balance the shrinkage parameter against the number of estimators.

Day 34

Advanced Boosting: XGBoost, LightGBM & CatBoost

XGBoost: Leverage second-order Taylor expansions and strict L1/L2 leaf regularization.
LightGBM: Achieve extreme speed via histogram-based binning and leaf-wise node growth.
CatBoost: Handle high-cardinality categorical targets natively using ordered boosting algorithms.
Implementation: Train APIs utilizing xgb.XGBClassifier() and lgb.LGBMClassifier().

Day 35

K-Means Clustering & Centroid Initialization

Lloyd's Algorithm: Alternate iteratively between centroid assignment and WCSS minimization.
Initialization: Prevent local-minima traps using probabilistic k-means++.
Metrics: Determine the optimal k using the visual Elbow Method.
Scaling: Deploy MiniBatchKMeans for memory-efficient multi-million row arrays.

Day 36

Hierarchical Clustering & Dendrograms + Density-Based Clustering (DBSCAN)

Agglomerative: Build bottom-up nested structures using AgglomerativeClustering.
Linkage Rules: Evaluate distances using Ward's, Complete, and Average linkage metrics.
Visualization: Map relationship hierarchies visually using dendrograms.
Cut-offs: Determine dynamic cluster counts by drawing horizontal thresholds across trees.
Arbitrary Shapes: Identify complex geometric clusters rather than forcing spherical assumptions.
Parameters: Define spatial density thresholds using eps and min_samples.
Noise: Natively isolate sparse anomalous data points (labeled as -1).
HDBSCAN: Extract variable-density clusters without strict parameter tuning.

Day 37

Gaussian Mixture Models (GMM)

Probabilistic Clustering: Model data as a mixture of multiple multivariate Gaussian distributions.
EM Algorithm: Optimize parameters via the Expectation-Maximization mathematical loop.
Soft Clustering: Return probability distributions for point assignments rather than hard labels.
Covariance Matrices: Allow for elliptical clusters using covariance_type='full'.

Day 38

Clustering Evaluation & Internal Metrics

Silhouette Score: Quantify separation distance between resulting clusters mathematically.
Davies-Bouldin Index: Evaluate intra-cluster similarity against inter-cluster differences.
Rand Index: Compare clustering outputs against known ground-truth labels if available.
Validation: Ensure unsupervised metrics align with EDA assumptions and domain knowledge.

Day 39

Dimensionality Reduction: PCA Mathematics + Manifold Learning: t-SNE & UMAP

Curse of Dimensionality: Understand the exponential volume expansion in high-dimensional space.
Covariance: Extract principal components via covariance matrix eigendecomposition.
Variance: Analyze Scree Plots using pca.explained_variance_ratio_.
Requirement: Enforce StandardScaler prior to PCA to ensure strict scale invariance.
t-SNE: Prevent spatial "crowding" by minimizing KL divergence via Student t-distributions.
UMAP: Preserve both local neighborhoods and global topological structures simultaneously.
Pipelines: Combine PCA for initial reduction before feeding into computationally heavy t-SNE.
Visuals: Map extremely high-dimensional ML outputs onto human-readable 2D/3D scatter plots.

Day 40

Anomaly & Outlier Detection Algorithms

Isolation Forest: Detect anomalies via shortest path lengths in randomized tree partitions.
Boundaries: Fit tight geometric hyperspheres around normal data using OneClassSVM.
Local Outlier Factor (LOF): Flag points with substantially lower density than their neighbors.
Pipelines: Automate the removal of mathematical anomalies before feeding data into linear models.

Month 03 — Advanced ML, Deep Learning & Vision

Days 41-60

Day 41

Time Series Forecasting Foundations

Components: Decompose data into Trend, Seasonality, Cyclical, and Noise elements.
Stationarity: Validate constant mean/variance via Augmented Dickey-Fuller tests (adfuller).
Modeling: Forecast sequential data using ARIMA, SARIMA, and Facebook Prophet.
Validation: Enforce temporal order via TimeSeriesSplit to prevent look-ahead bias.

Day 42

Automated Hyperparameter Tuning (Optuna) + Model Interpretability: SHAP & LIME

Grid vs Random: Compare exhaustive grid mapping against stochastic RandomizedSearchCV.
Bayesian Optimization: Navigate mathematical loss surfaces intelligently utilizing Gaussian Processes.
Optuna Framework: Build dynamic objective functions utilizing optuna.create_study().
Pruning: Abort unpromising ML training trials early using the Median Pruning algorithm.
The Black Box: Break down complex tree ensembles into human-understandable rules.
SHAP Values: Calculate precise marginal feature contributions based on game theory mathematics.
Visualization: Plot shap.summary_plot and shap.force_plot for individual predictions.
LIME: Fit interpretable linear surrogate models around specific localized predictions.

Day 43

Advanced Interpretability: PDP & ICE

Partial Dependence: Isolate and plot the marginal effect of one or two features on the predicted outcome globally.
ICE Plots: Use Individual Conditional Expectation to map feature relationships strictly at the instance level.
Feature Interactions: Identify non-linear interaction terms programmatically utilizing Friedman's H-statistic.
Dashboarding: Generate interactive interpretability reports natively using ExplainerDashboard.

Day 44

Auto-ML Frameworks & Meta-Learning

PyCaret / TPOT: Automate entire ML lifecycles from data imputation to model ensembling mathematically.
Search Spaces: Define robust search boundaries for genetic algorithms to find optimal pipeline architectures.
H2O.ai: Build extreme gradient boosting and deep learning ensembles via automated leaderboards.
Efficiency: Drastically reduce boilerplate modeling time to focus heavily on advanced feature engineering.

Day 45

Advanced Validation & Imbalance Calibration + Capstone: End-to-End ML Pipeline

Learning Curves: Diagnose Bias vs. Variance explicitly by plotting performance against training set sizes.

Probability Calibration: Correct overconfident model probabilities natively via Isotonic Regression or Platt Scaling.

Custom Metrics: Define custom scorer functions in scikit-learn strictly tied to real-world business KPIs.

Threshold Tuning: Shift default 0.5 decision thresholds explicitly to optimize the Precision/Recall tradeoff.

The Project: Build an end-to-end classification/regression system predicting a high-value metric.

Engineering: Combine Pandas pipelines, Target Encoding, SMOTE, and XGBoost into an sklearn.pipeline.

Evaluation: Validate via robust ROC-AUC tracking, Learning Curves, and SHAP interpretability reports.

Output: Serialize the final, fully optimized champion model object.

Day 46

TensorFlow Foundations, Tensors & GPU Compute

Tensors: Initialize arrays via tf.zeros(), tf.random.normal(), and tf.constant().

Execution: Understand Eager Execution vs. static Graph compilation utilizing @tf.function for speed.

Device Management: Map operations to hardware explicitly utilizing with tf.device('/GPU:0'): and configure memory growth.

Gradient Tape: Automate reverse-mode differentiation fundamentally by recording operations within tf.GradientTape().

Day 47

The Perceptron, Dense Layers & Keras API

Mechanics: Compute standard dot products via $\hat{y} = \sigma(w^T x + b)$.

Limitations: Understand mathematically why single layers fail on non-linear XOR distributions.

Sequential API: Stack linear transformations rapidly utilizing tf.keras.Sequential and tf.keras.layers.Dense.

Theory: Prove network capacity via the Universal Approximation Theorem.

Day 48

Backpropagation & Computation Graphs + Activation Functions & Gradient Flow

Calculus: Apply the chain rule explicitly across TensorFlow Directed Acyclic Graphs (DAGs).

Routing: Understand how gradients sum at addition nodes and switch at multiplications.

Instability: Identify the mathematical causes of Vanishing and Exploding gradients.

Clipping: Prevent parameter explosion dynamically natively within optimizers (e.g., clipnorm, clipvalue).

Saturating: Analyze why Sigmoid and Tanh destroy gradients in deep layer architectures.

ReLU: Fix saturation using max(0,x) via tf.keras.activations.relu while mitigating "dying neurons".

Modern Variants: Implement tf.keras.layers.LeakyReLU, PReLU, and exponential ELU adjustments as distinct layers.

GELU: Apply Gaussian Error Linear Units (used heavily in Transformers) for state-of-the-art flow.

Day 49

Model Compilation, Losses & Custom Training

Loss Functions: Optimize MeanSquaredError for regression and CategoricalCrossentropy for classification.

Optimizers: Transition from basic tf.keras.optimizers.SGD to adaptive methods like AdamW.

Batch Normalization: Stabilize deep networks using tf.keras.layers.BatchNormalization to re-center activations to zero-mean.

Training Loop: Compare high-level model.fit() against low-level custom loops: tape.gradient() → optimizer.apply_gradients().

Day 50

Image Tensors, tf.data & Feature Extraction

Tensors: Treat images as 3D matrices [Height, Width, Channels] typical in TensorFlow.

I/O: Read, decode, and display images natively utilizing tf.io.read_file() and tf.image.

Data Pipelines: Build highly optimized, asynchronous data loaders utilizing tf.data.Dataset.

Transforms: Prepare input tensors utilizing tf.keras.layers.Rescaling(1./255) and mapping functions.

Day 51

Spatial Filters, Kernels & Convolutions + Pooling Layers & Receptive Fields

Mechanics: Understand how sliding kernels compute local dot products using tf.keras.layers.Conv2D().
Invariance: Achieve translation invariance through shared spatial kernel weights.
Hyperparameters: Control output dimensions using strides and 'valid'/'same' padding arguments.
Math: Calculate spatial shapes using the strict formula: ⌊(W − K + 2P) / S⌋ + 1.
Downsampling: Compress spatial dimensions to aggressively reduce mathematical parameter load.
Methods: Implement tf.keras.layers.MaxPool2D() for sharp feature retention and AveragePooling2D.
Receptive Field: Calculate how deep layer pixels logically map back to large regions of the original image.
Flattening: Transition from 2D spatial maps to 1D vectors via tf.keras.layers.Flatten().

Day 52

Deep Architectures: VGG & Inception

VGG: Build deep, uniform architectures utilizing strictly 3x3 kernels.
Inception: Capture multiple spatial scales simultaneously using parallel filter branches.
Efficiency: Deploy 1x1 convolutions explicitly for channel-wise dimensionality reduction.
Transfer Learning: Load pre-trained weights utilizing tf.keras.applications.VGG16 and adapt classification heads.

Day 53

Residual Networks (ResNet) & Skip Connections

Degradation: Analyze why ultra-deep networks natively suffer from vanishing gradients.
Skip Connections: Implement residual blocks where output is F(x) + x to provide gradient highways via Functional API.
Bottlenecks: Optimize deep ResNets using 1x1 convolutions to compress and expand computational channels.
Implementation: Fine-tune tf.keras.applications.ResNet50(weights='imagenet') specifically on custom image data.

Day 54

Efficient Architectures: MobileNet & EfficientNet + Two-Stage Object Detectors (R-CNN Family)

Edge AI: Adapt complex vision models for strict mobile/IoT computational budgets.
MobileNet: Split standard convolutions utilizing Depthwise Separable Convolutions.
EfficientNet: Scale models uniformly across depth, width, and resolution using compound constants.
Quantization: Compress memory weights from 32-bit floats down to 8-bit integers.
Evolution: Trace the history from R-CNN to Fast R-CNN utilizing RoI pooling.
Faster R-CNN: Replace selective search with a fully integrated Region Proposal Network (RPN).
Anchors: Generate fixed-size anchor boxes mathematically to regress exact object boundaries.
Metrics: Evaluate models using Intersection over Union (IoU) and Mean Average Precision (mAP).

Day 55

Single-Stage Detectors (YOLO architectures)

Real-Time: Frame object detection as a single holistic regression problem for extreme speed.
Grid Division: Divide images into an S×S grid predicting bounding boxes directly.
NMS: Filter overlapping prediction boxes utilizing Non-Maximum Suppression algorithms.
Deployment: Utilize the ultralytics Python library to train and deploy YOLOv8/v10 pipelines.

Day 56

Semantic & Instance Segmentation

Semantic (FCN/U-Net): Classify every pixel natively utilizing a symmetric encoder-decoder structure.
Instance (Mask R-CNN): Distinguish between overlapping objects of the identical class.
Mask Branch: Add a parallel fully convolutional branch to generate precise binary masks.
Losses: Optimize utilizing Pixel-wise Cross Entropy and mathematical Dice Loss.

Day 57

Advanced Data Augmentation (Albumentations) + Vision Transformers (ViT)

Overfitting Defense: Synthetically expand small datasets to force model generalizability.
Transforms: Apply robust pipelines using the high-performance albumentations library.
Photometric: Inject Gaussian noise, modify HSV, and adjust brightness/contrast dynamically.
Occlusion: Force networks to learn holistic structures by dropping rectangular patches via Cutout.
Paradigm Shift: Discard convolutions entirely for self-attention mechanisms.
Tokenization: Split images into flattened 16x16 patches, linearly embedded as sequential tokens.
Global Context: Process patches through standard Transformers for immediate global image context.
Class Token: Prepend a learnable [CLS] token to aggregate image-level classification features.

Day 58

State-of-the-Art Augmentation: Mixup & CutMix

Mixup: Blend two different images and their one-hot labels linearly for smooth boundaries.
CutMix: Paste a rectangular patch from one image directly onto another distinct image.
Synergy: Combine the regularization of spatial Dropout with the label smoothing of Mixup.
Application: Apply these techniques natively within custom tf.data map functions.

Day 59

Deep Learning Loss Functions

Class Imbalance: Apply Focal Loss to down-weight easy examples and hyper-focus on hard mistakes.
Segmentation Loss: Implement Dice Loss and IoU Loss to maximize exact boundary overlaps.
Metric Learning: Use Triplet Margin Loss to build similarity embeddings natively.
Custom Loss: Subclass tf.keras.losses.Loss to author complex, multi-objective loss formulas.

Day 60

Transfer Learning & Parameter Freezing + Capstone: Custom CV Object Detection Pipeline

Strategies: Determine when to fine-tune the entire network versus only the final classifier head.
Freezing: Explicitly disable gradient calculations using layer.trainable = False.
Learning Rates: Apply Differential Learning Rates or Cosine Decay scheduling explicitly.
Callbacks: Implement EarlyStopping and ModelCheckpoint natively in Keras.
Data Handling: Parse complex bounding box XML/JSON formats into a custom TensorFlow pipeline.
Augmentation: Construct a rigorous Albumentations bounding-box-aware augmentation pipeline.
Modeling: Train a custom YOLOv10 model via Transfer Learning on a specialized dataset.
Evaluation: Calculate mAP scores and run inference scripts plotting boxes directly via cv2.rectangle().

Month 04 — NLP, Transformers & Generative AI Intro

Days 61–80

Day 61

Text Normalization & Regex Pipelines

Cleaning: Standardize casing, remove HTML markup, and expand linguistic contractions.
Regex Operations: Extract and sanitize exact patterns utilizing re.sub() and re.findall().
Noise Reduction: Strip special characters and URLs to prevent deep learning embedding distortion.
Unicode: Handle text encoding anomalies securely utilizing the unicodedata library.

Day 62

Lemmatization, POS Tagging & NER (spaCy)

Stemming vs Lemmatization: Compare algorithmic chopping against linguistic dictionary roots via spaCy.
Stop Words: Filter low-information structural words to condense vocabularies.
POS Tagging: Identify syntactic roles natively (Nouns, Verbs) using sequence taggers.
Named Entity Recognition: Extract geopolitical entities and names automatically via en_core_web_sm.

Day 63

Tokenization Algorithms & Subword Paradigms + Bag of Words (BoW) & TF-IDF Vectorization

Word Tokenization: Understand the massive vocabulary limits of naive .split() operations.
Subword: Solve Out-Of-Vocabulary (OOV) problems by mathematically fragmenting unknown words.
Algorithms: Detail Byte-Pair Encoding (BPE), WordPiece (BERT), and SentencePiece architectures.
Implementation: Train custom tokenizers natively using tokenizers.ByteLevelBPETokenizer.
Sparse Matrices: Convert raw text into numerical tensors utilizing frequency occurrences.
Count Vectorizer: Implement n-gram counting via sklearn...CountVectorizer.
TF-IDF Formula: Weigh term importance mathematically (Term Frequency × Inverse Document Frequency).
Application: Feed sparse TF-IDF outputs into Naive Bayes for baseline document classification.

Day 64

Dense Word Embeddings (Word2Vec / GloVe)

Continuous Space: Map semantic word meaning explicitly to high-dimensional geometric proximity.
Word2Vec: Train shallow networks utilizing Continuous Bag of Words (CBOW) and Skip-Gram methods.
GloVe: Factorize global co-occurrence matrices for contextual mathematical embeddings.
Libraries: Train domain-specific word vectors natively utilizing the gensim.models package.

Day 65

Markov Assumptions & n-gram Language Models

Pre-Neural Modeling: Predict the next word using strict probabilistic conditional counts.
Markov Property: Assume context strictly depends only on a fixed preceding window.
Sparsity: Implement Laplace smoothing mathematically to prevent zero-probability crashes.
Perplexity: Evaluate LM quality by calculating the inverse probability of an unseen test set.

Day 66

Recurrent Neural Networks (RNN) & BPTT + Vanishing & Exploding Gradients Analysis

Recurrence: Process sequential data by maintaining an internal hidden state vector utilizing tf.keras.layers.SimpleRNN.
Weight Sharing: Apply the exact same weight matrix natively to every element in a temporal sequence.
BPTT: Mathematically unroll the network to sum gradient contributions across time steps.
Padding: Handle variable-length text batches via tf.keras.preprocessing.sequence.pad_sequences.
The Flaw: Analyze how continuous matrix multiplication causes exponential decay or explosions.
Amnesia: Prove mathematically why standard RNNs cannot retain long-term linguistic dependencies.
Mitigation: Implement strict gradient clipping explicitly within TensorFlow optimizers (clipnorm=1.0).
Initialization: Deploy orthogonal weight initialization to preserve eigen-values temporarily.

Day 67

Long Short-Term Memory (LSTM) Networks

Cell States: Establish a persistent "memory highway" unhindered by non-linearities.
Forget Gate: Determine what context to discard using a sigmoid layer (f_t).
Input/Output Gates: Regulate new information addition and extraction into the hidden state.
Keras: Implement multi-layer stacked LSTMs natively utilizing tf.keras.layers.LSTM(return_sequences=True).

Day 68

Gated Recurrent Units (GRU) & Bi-Directional Models

GRU Simplification: Merge forget and input gates into a single 'Update Gate' for computational speed.
Implementation: Substitute LSTMs with tf.keras.layers.GRU while maintaining sequence retention parity.
Bi-Directionality: Process sequences forwards and backwards simultaneously via tf.keras.layers.Bidirectional.
Concatenation: Merge the hidden states of both directions natively to enrich contextual representation.

Day 69

Sequence-to-Sequence (Seq2Seq) Translators + The Information Bottleneck Problem

Asymmetric Generation: Map variable-length inputs natively to variable-length outputs.
Encoder: Compress the entire source sequence into a final, fixed-size hidden Context Vector.
Decoder: Autoregressively generate output sequences, conditioned strictly on the Encoder's state.
Teacher Forcing: Accelerate training by feeding ground-truth tokens as next-step decoder inputs.
Architectural Failure: Analyze why a single, fixed-size vector fails to capture massive document nuances.
Degradation: Observe BLEU score collapse mathematically on sentences longer than 20 words.
Reversing Inputs: Discuss classic workarounds (feeding sentences backwards) to shorten dependencies.
Necessity: Establish the theoretical necessity for a dynamic, variable routing mechanism.

Day 70

Introduction to Attention Mechanisms

The Breakthrough: Allow the Decoder to dynamically "look back" at ALL Encoder hidden states.
Dynamic Context: Calculate a unique, weighted sum of encoder states specifically for each generated token.
Alignment: Train an internal feed-forward network natively to score semantic relevance between states.
Interpretability: Plot attention weight matrices natively to physically map translation focus.

Day 71

QKV (Query, Key, Value) Alignments

Soft Attention: Calculate differentiable attention distributions over all tokens using Softmax.
QKV Framework: Introduce the Query (decoder state), Key (encoder states), and Value (encoder outputs) mental model.
Dot Product: Compute rapid similarity alignment using pure, highly-optimized matrix multiplication QK^T.
Masking: Inject -inf into pad token locations to nullify their mathematical attention weights.

Day 72

Neural Machine Translation & BLEU Metrics + Self-Attention & Parallel Processing

Pipeline: Assemble complete end-to-end NMT architectures substituting phrase-based statistics.
BLEU Score: Evaluate text generation mathematically against references using n-gram overlap penalties.
ROUGE Score: Calculate recall-oriented evaluation metrics natively for summarization tasks.
Shortcut Gradients: Prove how Attention implicitly bypasses vanishing gradients by shortening logic paths.
Attention Is All You Need: Discard RNN recurrence entirely; process sequential tokens fully in parallel.
Self-Attention: Contextualize words explicitly by attending to other words within the same sentence.
Linear Projections: Project token embeddings into Query, Key, and Value matrices.
Math: Execute the core formula: $\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k})V$.

Day 73

Multi-Head Attention & Scaled Dot-Products

Scaling Factor: Prevent large dot-product magnitudes from pushing Softmax into zero-gradient zones.
Multi-Head: Deploy parallel attention heads (MultiHeadAttention) to capture divergent linguistic rules.
Concatenation: Merge parallel head outputs natively and project via a final linear weight matrix.
Residuals: Apply LayerNorm and residual connections securely around every block.

Day 74

Positional Encoding & Causal Masking

Injection: Resolve order-invariance mathematically by injecting sinusoidal waves into input embeddings.
RoPE: Implement modern Rotary Positional Embeddings natively utilized in architectures like Llama.
Causal Masks: Implement lower-triangular matrices to block autoregressive decoders from "seeing the future".
Integration: Pass boolean masks securely into the attention algorithms.

Day 75

Transformer Encoders (BERT structure) + Capstone: NMT Transformer from Scratch

Bidirectional Context: Analyze BERT's architecture using strictly the Transformer Encoder stack.
Pre-training: Master Masked Language Modeling (MLM) and Next Sentence Prediction (NSP) objectives.
HuggingFace: Load transformers.TFBertModel and utilize its [CLS] token natively for classification.
Fine-Tuning: Swap the classification head and execute standard gradient updates on downstream tasks.
From Scratch: Construct a complete multi-head attention mechanism using primitive tf.linalg.matmul operations.
Architecture: Assemble Encoder and Decoder blocks including Feed-Forward networks and LayerNorms.
Data Handling: Implement Byte-Pair Encoding and generate complex causal/padding tensors.
Task: Train the scratch Transformer strictly on a Machine Translation dataset and visualize attention matrices.

Day 76

Transformer Decoders (GPT Family)

Autoregressive Generation: Build causal models where token N exclusively predicts token N+1.
Scaling Laws: Analyze mathematical correlations between parameter count, dataset size, and compute budgets.
Emergent Abilities: Understand zero-shot reasoning capabilities manifesting in models exceeding 10B parameters.
KV Cache: Optimize inference memory structurally by caching previously computed Key and Value vectors.

Day 77

Decoding Strategies & Output Entropy

Deterministic: Compare Greedy Search against the branching probabilistic logic of Beam Search.
Stochastic Sampling: Implement Top-K (fixed list) and Nucleus Top-p (cumulative probability) constraints.
Temperature: Adjust softmax T: T > 1 flattens distribution (creative); T < 1 sharpens it (factual).
Penalties: Tune frequency_penalty and presence_penalty natively to minimize generative repetition.

Day 78

Advanced Prompt Engineering (In-Context) + Chain/Tree of Thought Reasoning

Anatomy: Construct strict system prompts defining Persona, Context, Constraints, and Target schemas.
In-Context Learning: Execute Zero-Shot, One-Shot, and Few-Shot conditioning without altering model weights.
Structured Output: Force generations into strict computational structures utilizing JSON response formats.
Guardrails: Design robust system boundaries proactively to mitigate adversarial prompt injection attacks.
CoT: Mitigate hallucinations by prompting the model to generate step-by-step intermediate reasoning traces.
ToT: Expand logic into non-linear heuristic search trees, evaluating multiple potential conversational branches.
Self-Consistency: Generate multiple CoT paths and logically aggregate the most frequently deduced factual answer.
Application: Deploy these algorithms strictly for multi-step math, logic, and coding generation tasks.

Day 79

ReAct (Reason and Act) Frameworks

Agentic Bridge: Interleave internal logic processing directly with external computational tool actions.
The Loop: Force the model into a strict Thought → Action → Observation cyclical pipeline.
Tools: Equip the prompt securely with simulated API definitions (Calculator, Web Search, SQL query).
Autonomy: Build a while loop allowing the LLM to recursively issue commands until final resolution.

Day 80

Embeddings & Vector Databases

The Architecture: Ground generative language models dynamically in proprietary, private datasets.
Embedding Models: Convert text into dense 1536-dimensional vectors utilizing text-embedding-3.
Vector DBs: Provision and ingest vectors securely into databases like Pinecone, ChromaDB, or pgvector.
Indexing: Understand the math behind HNSW for rapid Approximate Nearest Neighbor retrieval.

Month 05 — Advanced GenAI, RAG, & Agentic Systems

Days 81–100

Day 81

Semantic Chunking & Document Processing + Query Transformation (HyDE)

Loaders: Extract unstructured data via specialized PDF, HTML, and Markdown document parsers.
Recursive Splitting: Implement RecursiveCharacterTextSplitter with explicit sliding chunk overlaps.
Semantic Chunking: Split paragraphs dynamically by monitoring cosine distance shifts between sentences.
Metadata: Preserve parent-child document relationships logically to feed wider context post-retrieval.
HyDE: Implement Hypothetical Document Embeddings—prompt the LLM to hallucinate an answer, then embed it.
Query Expansion: Utilize the LLM to mathematically break complex queries into multiple smaller sub-queries.
Metadata Filtering: Tag document chunks securely to pre-filter retrieval pools based on hard constraints.
Self-Querying: Build routers that autonomously convert natural language into structured Metadata + Vector filters.

Day 82

Hybrid Search & Cross-Encoder Reranking

Hybrid Search: Combine Dense semantic vector search natively with Sparse keyword-matching (BM25).
Fusion: Merge disparate retrieval lists utilizing the Reciprocal Rank Fusion (RRF) mathematical algorithm.
Cross-Encoders: Feed [Query + Document] into a classification model for absolute precision scoring.
Pipeline: Retrieve Top-100 efficiently via Vectors, then logically sort the Top-10 via the Cross-Encoder.

Day 83

GraphRAG & Knowledge Graphs

Graph Theory: Represent complex corporate data via strict Nodes (Entities) and Edges (Relationships).
Extraction: Use LLMs to autonomously perform unstructured text-to-graph translation pipelines.
Neo4j: Store and traverse connected data utilizing the Cypher graph query language.
Multi-Hop: Deploy GraphRAG natively to answer complex relational questions vector similarity fails to connect.

Day 84

Orchestration Frameworks: LangChain & LlamaIndex + Self-RAG & Corrective RAG (CRAG)

LangChain: Connect prompts, models, and parsers utilizing strict pipeline syntax (LCEL).
Memory Modules: Maintain contextual chat history securely using ConversationBufferMemory.
LlamaIndex: Build specialized Data Connectors and Query Engines optimized explicitly for RAG systems.
Routing: Implement semantic routers to dynamically direct queries to the appropriate vector/SQL database.
CRAG: Design an evaluator pipeline where an LLM explicitly grades the relevance of retrieved vector documents.
Fallback Mechanics: Automatically trigger external Web Search APIs if internal document grading fails entirely.
Self-RAG: Train models to output distinct [Critique] tokens assessing if generation is factually supported.
Graphs: Orchestrate cyclic, stateful decision flows securely utilizing the LangGraph framework.

Day 85

Tool Calling & Model Context Protocol (MCP)

Native Tool Use: Bind Python functions dynamically to LLM inference utilizing strict JSON Schema declarations.
Execution: Safely parse the LLM's requested parameters, execute the local function, and return the tool_message.
MCP: Connect AI models securely to enterprise filesystems, Slack, and GitHub via standardized protocols.
HITL: Implement "Human-in-the-Loop" manual approval breakpoints explicitly for destructive agentic actions.

Day 86

Parameter-Efficient Fine-Tuning (PEFT/LoRA)

The Cost: Understand the immense computational/VRAM requirements for full-weight Supervised Fine-Tuning.
LoRA Math: Freeze base weights and inject trainable low-rank decomposition matrices (A and B) into attention layers.
QLoRA: Execute 4-bit NormalFloat quantization to fine-tune 7B parameter models securely on a single GPU.
Libraries: Deploy the HuggingFace peft and trl libraries to manage training loops natively.

Day 87

Preference Alignment (RLHF & DPO) + Application Frontends: Streamlit & Gradio

Alignment Theory: Ensure base completions map strictly to human behavioral preferences (Helpful, Honest, Harmless).
RLHF: Train a Reward Model on human preferences, then optimize policy utilizing Proximal Policy Optimization (PPO).
DPO: Direct Preference Optimization solves for optimal policy using simple classification loss directly on preference pairs.
Formatting: Prepare ChatML datasets representing strictly chosen vs. rejected outputs.
Streamlit: Convert pure Python pipelines instantly into interactive React-based web dashboards.
State Management: Manipulate st.session_state natively to store API keys, chat history, and context.
Chat UI: Deploy st.chat_message() interfaces structurally with live streaming text capabilities.
Gradio: Build rapid, shareable API-based prototypes explicitly for multimodal model inference.

Day 88

Agentic Orchestration & Workflows

Visual Node Engines: Use n8n or Flowise for low-code orchestration of complex multi-branch workflows.
Webhooks: Integrate generative pipelines directly into SaaS tools via external API triggers.
Serverless Agents: Deploy backend Python execution APIs natively via AWS Lambda or Google Cloud Run.
Async Task Queues: Manage long-running agent tasks gracefully utilizing Celery and Redis.

Day 89

LLM Evaluation Metrics (RAGAS / DeepEval)

Stochastic Testing: Transition explicitly from deterministic Unit Tests to probabilistic evaluation rubrics.
Metrics: Calculate strict metrics for Context Precision, Context Recall, Faithfulness, and Answer Relevance.
LLM-as-a-Judge: Automate evaluations utilizing a superior LLM scoring generation outputs based on complex rubrics.
CI/CD: Integrate automated evaluation scripts seamlessly into GitHub Actions prior to production deployment.

Day 90

Execution Tracing & Data Drift + Capstone: Autonomous Multi-Agent System

Execution Tracing: Map complex nested agentic API calls visually utilizing platforms like LangSmith.
Latency: Profile token burn rates, TTFT (Time To First Token), and bottleneck spans systematically across chains.
Prompt Management: Version control prompt templates securely, independently of core codebase application logic.
Semantic Drift: Monitor production vector databases dynamically for mathematical shifts in user queries over time.
The Project: Architect and deploy a production-grade Autonomous Research Agent.
System: Integrate Hybrid Search RAG, LangGraph Multi-Agent routing, and dynamic Python REPL APIs.
Interface: Build an async streaming Streamlit interface supported by a secure FastAPI backend.
Monitoring: Complete with deep LangSmith observability and automated RAGAS evaluations.

Day 91

Multimodal Models & Vision-Language

CLIP: Understand Contrastive Language-Image Pretraining architecture.
Embeddings: Align text and image vectors into a shared mathematical space.
Generation: Deploy visual-question answering pipelines using specialized LLMs.

Day 92

AI Ethics, Bias Mitigation & Fairness

Disparate Impact: Identify and measure hidden bias in historical training datasets.
Fairness Metrics: Calculate Equal Opportunity and Demographic Parity natively.
Red Teaming: Actively probe generative models for unsafe or restricted outputs.

Day 93

Model Security & Adversarial Robustness + Federated Learning Foundations

Attacks: Understand Fast Gradient Sign Method (FGSM) and prompt injection.
Defense: Train models with adversarial examples to increase robustness boundaries.
Sanitization: Scrub inbound user prompts securely before reaching the inference engine.
Decentralization: Train models collaboratively without centralizing raw user data.
Aggregation: Apply Federated Averaging mathematically to update global weights.
Use Cases: Analyze implementations in healthcare and cross-device personalization.

Day 94

ML System Design Frameworks

Framework: Master the 5-step approach (Requirements, Metrics, Architecture, Data, Scale).
Trade-offs: Justify architectural decisions (Batch vs Streaming, CPU vs GPU).
Communication: Lead whiteboarding discussions regarding single points of failure.

Day 95

Capstone Phase 1: Problem Scoping

Ideation: Define a high-impact business problem solvable via Advanced ML.
Requirements: Draft strict technical specifications and success metrics.
Sourcing: Identify, acquire, and validate necessary raw datasets.

Day 96

Capstone Phase 2: Data & Modeling + Capstone Phase 3: Deployment Architecture

Pipelines: Construct automated EDA and feature engineering loops.
Training: Train, tune, and log champion models utilizing robust cross-validation.
Validation: Defend model selection based on mathematical metrics.
API: Wrap the final serialized model natively in a strictly typed FastAPI container.
Local Run: Ensure seamless local execution and API endpoint pinging.
Documentation: Finalize README.md, architecture diagrams, and comprehensive API docs.

Day 97

Capstone Phase 4: UI/UX & Polish

Frontend: Build a highly interactive frontend utilizing Streamlit or Gradio.
User Experience: Ensure latency remains low and error messages are clear.
Portfolio: Curate a professional, production-ready GitHub repository.

Day 98

Capstone Presentation & Industry Graduation

Pitch: Deliver a 15-minute technical presentation to stakeholders.
Q&A: Defend architectural, systemic, and mathematical choices under intense scrutiny.
CORE GRADUATION: Officially conclude the core AIML engineering program!

Day 99

Big-O & Problem Solving Mindset + Arrays & Memory Basics

Complexity: Analyze basic Time & Space Complexity (O(1), O(n), O(log n), $O(n^2)$).
Relevance: Understand why efficiency matters for large ML datasets.
Mindset: Develop pattern recognition skills to map coding problems logically.
Data Structures: Understand Python lists as dynamic arrays in memory.
Techniques: Implement foundational Two-Pointer techniques for traversal.
Problems: Solve basic array problems like Two Sum and reversing an array.

Day 100

String Manipulation Logic

Immutability: Understand how string manipulation affects memory.
Techniques: Apply the Sliding Window technique to find substrings.
Problems: Find longest non-repeating substrings and valid palindromes.

Month 06 — Capstone Conclusion, DSA, SQL & MLOps

Days 101–120

Day 101

Basic Searching Algorithms

Linear Search: Understand standard O(N) iterative searching.
Binary Search: Implement O(log N) search on sorted data.
Intuition: Relate binary search concepts to hyperparameter space halving.

Day 102

Basic Sorting Algorithms + Linked Lists Foundations

Algorithms: Understand Bubble Sort, Selection Sort, and basic Merge Sort.
Time/Space: Compare the $O(N^2)$ worst cases versus O(N log N) optimization.
Python API: Utilize Python's internal sort() and sorted() efficiently.
Structures: Understand Singly Linked List nodes and pointers.
Techniques: Traverse, insert, and delete nodes without native array indexing.
Problems: Reverse a linked list manually and detect basic cycles.

Day 103

Stacks & Queues

Structures: Understand LIFO (Stacks) and FIFO (Queues) principles.
Implementation: Build stacks using lists and queues using collections.deque.
Applications: Solve valid parentheses and basic sequential tracking problems.

Day 104

Hashmaps & Sets

Dictionaries: Deep dive into internal hashing mechanics and O(1) lookups.
Sets: Perform fast containment checks and deduplication explicitly.
Problems: Utilize frequency counting (collections.Counter) for rapid math.

Day 105

Introduction to Recursion + Basics of Binary Trees

Mechanics: Understand base cases and map basic execution flows.
Memory: Visualize Call Stack execution during deep recursive functions.
Problems: Calculate factorials and the Fibonacci sequence recursively.
Structures: Build basic binary tree nodes and link children.
Traversals: Implement Inorder, Preorder, and Postorder traversals natively.
Search: Understand the basic properties of a Binary Search Tree (BST).

Day 106

SQL Fundamentals and DDL

RDBMS & SELECT: Understand the relational model. Master SELECT and WHERE filtering.
DDL: Structure databases: CREATE TABLE, ALTER TABLE, and DROP TABLE.
Constraints: Enforce integrity with Primary/Foreign Keys, UNIQUE, and NOT NULL.

Day 107

Data Manipulation & Retrieval

DML: Insert (INSERT INTO), update (UPDATE), and delete (DELETE) data.
Transactions: Maintain consistency using COMMIT and ROLLBACK.
DQL: Use ORDER BY for sorting and DISTINCT to extract unique values.

Day 108

Aggregation & Grouping + Advanced Filtering & Set Operations

Math: Utilize COUNT, SUM, and AVG for column-level calculations.
Extremes: Use MIN and MAX to find extremes within datasets.
Grouping: Aggregate distinct categories via GROUP BY and filter them using HAVING.
Filtering: Apply logical operators (AND, OR) and special operators (LIKE, IN, BETWEEN).
Set Operators: Combine multiple queries using UNION, UNION ALL, INTERSECT, and EXCEPT.
Execution Order: Understand how SQL conceptually executes written commands.

Day 109

Relational Data Joins

Inner/Outer Joins: Master INNER JOIN, LEFT, RIGHT, and FULL OUTER JOIN handling unmatched rows.
Advanced Joins: Explore non-equi joins, CROSS JOIN (Cartesian), and SELF-JOIN.
Architecture: Understand why normalized relational schemas require heavy joining.

Day 110

Built-in Functions & Null Handling

Strings: Manipulate text (SUBSTRING, LENGTH, UPPER, TRIM).
Nulls: Address missing data securely using IS NULL and COALESCE/ISNULL.
Dates: Extract parts, calculate differences (DATEDIFF), and perform date arithmetic.

Day 111

Conditional Logic & Subqueries + CTEs & Procedures

Aliases: Use column and table ALIASES to write cleaner SQL code.
Logic: Implement conditional mapping natively using the CASE expression.
Subqueries: Write independent subqueries and use EXISTS for correlated subqueries.
Procedures: Encapsulate basic SQL logic strictly inside Stored Procedures.
CTEs: Define named, temporary result sets (WITH clause) to simplify complex reads.
Recursion: Understand the concept of Recursive CTEs conceptually vs standard queries.

Day 112

Window Functions (Part 1)

Concept: Introduce calculations across a defined set of table rows dynamically.
Ranking: Master strict ranking functions: RANK, DENSE_RANK, and ROW_NUMBER.
Application: Rank distinct rows without grouping them down into single aggregated lines.

Day 113

Window Functions (Part 2)

Partitions: Use PARTITION BY to establish exact analytical window boundaries.
Analytics: Calculate running totals and lagged/leaded values using LAG/LEAD.
Time-Series: Utilize moving averages natively within SQL for financial/data analysis.

Day 114

Introduction to MLOps + Data & Code Versioning

Lifecycle: Map the transition from local Jupyter Notebooks to strict production pipelines.
Concepts: Understand Continuous Integration, Continuous Delivery, and Continuous Training (CI/CD/CT).
Necessity: Discuss why models degrade over time and require operational supervision.
Code: Overview foundational Git commands (commit, push, branch) for ML scripts.
Data Control: Understand the strict necessity of Data Version Control (DVC) conceptually.
Reproducibility: Ensure environments and datasets can be mirrored by any other engineer.

Day 115

Experiment Tracking Basics

Tracking: Track hyperparameter choices, metric outputs, and random seeds programmatically.
Comparison: Understand how comparing isolated experiments prevents "lost" champion models.
Metadata: Log strict metadata associated with every specific training run.

Day 116

Introduction to MLflow

Server: Launch and navigate a local MLflow tracking server dashboard.
Logging: Utilize mlflow.log_param and mlflow.log_metric inside Python training loops.
Autolog: Implement mlflow.autolog() for rapid, out-of-the-box framework tracking.

Day 117

Model Serialization & Packaging + Introduction to REST APIs

Serialization: Save and load trained ML models natively utilizing pickle and joblib.
Pipelines: Bundle preprocessors (scalers/encoders) securely with the model object via sklearn.pipeline.
Dependencies: Extract a precise requirements.txt file to prevent critical version drift.
Concepts: Understand client-server architecture, HTTP verbs (GET/POST), and status codes.
JSON: Map Python dictionaries natively into structured JSON payload transfers.
Inference: Translate local model.predict() logic into a networked request-response cycle.

Day 118

Building Basic APIs with FastAPI

Framework: Initialize an asynchronous FastAPI application locally.
Endpoints: Create a strict @app.post("/predict") route to ingest feature data.
Pydantic: Enforce inbound JSON data structures mathematically using Pydantic validation schemas.

Day 119

Testing & Validating the API

Swagger UI: Test the endpoint interactively via the automatically generated Swagger docs (/docs).
Postman/cURL: Ping the local ML server securely using external networking tools.
Error Handling: Return proper 400 Bad Request codes for malformed data inputs.

Day 120

Introduction to Docker + Writing a Dockerfile for ML

Containerization: Understand isolation constraints versus traditional heavy virtual machines.
Images/Containers: Differentiate between a static Docker Image and a running Container.
Commands: Use basic CLI tools like docker build, docker run, and docker ps.
Declarations: Write a basic Dockerfile explicitly tailored for ML (FROM python:3.10, COPY, CMD).
Execution: Build the image and run the isolated API container locally, mapping internal ports.
Cloud Intro: Discuss conceptual pathways for pushing this container to AWS/GCP for public access.

PROJECTS & CAPSTONES

7 things you'll build

DAY 15

End-to-End Extraction Pipeline

A multi-page scraper built with OOP, type hints and decorators, cleaned with Pandas and exported to SQLite or Parquet.

DAY 45

End-to-End ML Pipeline

Target encoding, SMOTE and XGBoost in an sklearn pipeline, validated with ROC-AUC, learning curves and SHAP.

DAY 60

Custom Object-Detection Pipeline

Bounding-box-aware Albumentations, a YOLOv10 model trained via transfer learning, evaluated with mAP.

DAY 75

Transformer NMT from Scratch

Multi-head attention from primitive matmuls, BPE, causal masks — trained on machine translation.

DAY 90

Autonomous Multi-Agent Research System

Hybrid-search RAG, LangGraph multi-agent routing, a Python REPL tool, FastAPI + Streamlit, LangSmith and RAGAS.

DAYS 95–98

Final Capstone & Graduation

Scope a business problem, model it, deploy it behind FastAPI with a polished UI, then defend it in a 15-minute pitch.

DAYS 114–120

Production ML API

MLflow experiment tracking, serialised pipelines, a validated FastAPI service and a Dockerised deployment.

Frequently asked questions

Is this program suitable for beginners?
It starts with Python foundations, but the pace and depth are engineering-grade — concurrency, CPython internals, Transformers from scratch and MLOps. Comfort with logical thinking and maths will help you get the most from it.

Which deep-learning framework is used?
TensorFlow and Keras — tensors, GradientTape, custom training loops, tf.data pipelines, CNNs, ResNets, transfer learning, YOLO, segmentation and Vision Transformers.

How deep does the GenAI coverage go?
Attention, QKV and positional encoding, building a Transformer from scratch, GPT-style decoding, advanced prompting and ReAct, vector databases, advanced RAG, MCP tool calling, LoRA/QLoRA fine-tuning and RLHF/DPO alignment.

Does it prepare me for interviews?
Yes. Month 6 covers data structures and algorithms — Big-O, searching, sorting, linked lists, stacks,

queues, hashmaps, recursion and trees — plus SQL and ML system design frameworks.

Will I learn to deploy models?
Yes — Git and DVC, MLflow tracking, serialisation, REST APIs with FastAPI and Pydantic, testing with Swagger and Postman, and Docker containers.

NeuralVarsity

Ready to start Applied AI & Machine Learning?

Apply online in under three minutes. An admissions advisor will contact you to discuss fit, schedules and next steps.

01

Apply online

Choose your program and share your background.

02

Talk to an advisor

A quick conversation about goals, fit and schedules.

03

Enrol & start building

Get set up and begin your first session.

APPLY ONLINE

neuralvarsity.ai/apply.html?program=applied-ai-and-machine-learning

WHATSAPP

Message us via the website

PHONE

See website contact page

EMAIL

See website contact page

Apply now →